JP Morgn's Quantitative Research

Task 1: Investigate and analyze price data

Introduction

This task focuses on the analysis and extrapolation of natural gas price data, based on a monthly snapshot from a market data provider. The goal is to understand existing price trends and predict future prices, which is critical for long-term storage contracts.

Current Data Source

The existing data comprises monthly snapshots of natural gas prices, representing the market price at the end of each calendar month. This data covers roughly the next 18 months and is integrated with historical prices in a time series database. You will be provided with this data in a CSV file format.

Objectives

- **Download** the monthly natural gas price data.
- **Analyze** the data, which spans from **October 31st, 2020**, to **September 30th, 2024**, to estimate the purchase price of gas at any given date.
- **Extrapolate** the price data for an additional year to provide indicative prices for longer-term storage contracts.
- **Develop a function** that takes a date as input and returns a price estimate.
- **Visualize** the data to identify patterns, such as seasonal trends, that influence natural gas prices. (Note: Market holidays, weekends, and bank holidays do not need to be accounted for).
- **Submit** your completed code.

Professional Context

This role frequently requires expertise in data analysis and machine learning. Python is a widely used and highly effective tool in quantitative research at JPMorgan Chase, particularly for complex tasks.

Technical Note

Example solutions for this program will be provided in Python. If Python is not installed on your system, you can utilize Jupyter Notebook online for code execution.

```
# Importings basic tools
import pandas as pd
import numpy as np
import seaborn as sns
import datetime as dt
import matplotlib.pyplot as plt
```

```
# imorting math models and tools
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from google.colab import files
import os
```

1. Start working by first importing the data

```
# Check if the data file already exists to avoid re-uploading every time
file_name = "Natural_Gas_Prices.csv"

if not os.path.exists(file_name):
    print(f"File '{file_name}' not found. Please upload the file.")
    uploaded = files.upload()
    if uploaded:
        # Assuming only one file is uploaded and using its name
        file_name = list(uploaded.keys())[0]
        print(f'User uploaded file "{file_name}" with length {len(uploaded[file_name])} l
    else:
        print("No file was uploaded.")
else:
    print(f"File '{file_name}' already exists.")


natural_gas_df = pd.read_csv(file_name)
```

Show hidden output

```
# data fie is uploade now show the data
natural_gas_df
```

Show hidden output

```
natural_gas_df_ORIGINAL = natural_gas_df
```

Show the main Plot.

```
# genrate the plot
# I choosed dates as x-axis and prices as y-axis
plt.plot(natural_gas_df['Dates'], natural_gas_df['Prices'])
```

Show hidden output

2. Analyize the JAnuary Monts prices Using Linar Regression

Because it shows linear growth year on year

```
# select only january month prices
natural_gas_df['Dates'] = pd.to_datetime(natural_gas_df['Dates'])

natural_gas_df['Month'] = natural_gas_df['Dates'].dt.month
natural_gas_df['Year'] = natural_gas_df['Dates'].dt.year

# declare the january dataframe obkect
natural_gas_jan = natural_gas_df[natural_gas_df['Month'] == 1]

# declare the plot object of the jan df obect
natural_gas_jan_plot = plt.plot(natural_gas_jan['Dates'], natural_gas_jan['Prices'])

# Show the JAnuary's Price data the Natural GAs
natural_gas_jan
```

Show hidden output

```
from sklearn.linear_model import LinearRegression

x = np.array(natural_gas_df[natural_gas_df['Month'] == 1]['Year'] ).reshape(-1, 1)
y = np.array(natural_gas_df[natural_gas_df['Month'] == 1] ['Prices'])

reg = LinearRegression().fit(x, y)
reg
```

```
LinearRegression
LinearRegression()
```

```
#
round(float(reg.predict([[2025]])), 4)
```

```
/tmp/ipykernel_960/2851572027.py:2: DeprecationWarning: Conversion of an array with ndim >
  round(float(reg.predict([[2025]])), 4)
13.2
```

3. Extrapolation for an extra year

3.1 Function to predict next year's instrument (natural gas) price

```
from sklearn.linear_model import LinearRegression

# function to predict next year's instrument (natural gas) price
def next_year_price(next_year):
  "Return a predicted natural ags prices for each month in the following year"

  price_list = []
  for i in np.arange(12):
    x = np.array(natural_gas_df[natural_gas_df['Month'] == 1+i]['Year'] ).reshape(-1, 1)
    y = np.array(natural_gas_df[natural_gas_df['Month'] == 1+i]['Prices'])
    reg = LinearRegression().fit(x, y)
    price_list.append(round(float(reg.predict([[next_year]])), 4))

  return price_list
```

```
# predict the next year's natural gas prices "gas_price25"
gas_price25 = next_year_price(2025)
np.array(gas_price25)
```

```
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
/tmp/ipykernel_960/1347388608.py:12: DeprecationWarning: Conversion of an array with ndim
  price_list.append(round(float(reg.predict([[next_year]])), 4))
array([13.2 , 12.8 , 13.25, 12.65, 12.08, 11.95, 12.25, 11.9 , 12.45,
       12.85, 13.46, 13.66])
```

3.2 Get last of each month of the next year

```
# Function to get next year's months(last date) in ascending order in an array
def get_last_of_each_month(year):
  dates_array = []
  current_date = dt.date(year, 12, 31)      # Start from the last day of the Year
  while current_date.year == year:

    dates_array.append(current_date.strftime('%Y-%m-%d'))
    month = current_date.month
    year = current_date.year

    # Move to the first day of the previous month
    current_date = current_date.replace(year=year, month=month, day=1)

    # Move back one day to get the last day of the current month
    current_date -= dt.timedelta(days=1)

  return dates_array[::-1]
```

```
dates_2025 = get_last_of_each_month(2025)
dates_2025
```

```
['2025-01-31',
 '2025-02-28',
 '2025-03-31',
 '2025-04-30',
 '2025-05-31',
 '2025-06-30',
 '2025-07-31',
 '2025-08-31',
 '2025-09-30',
 '2025-10-31',
 '2025-11-30',
 '2025-12-31']
```

3.3 EXTRAPOLATION: *Projects the next year's price of the natural gas on the last date of each month*

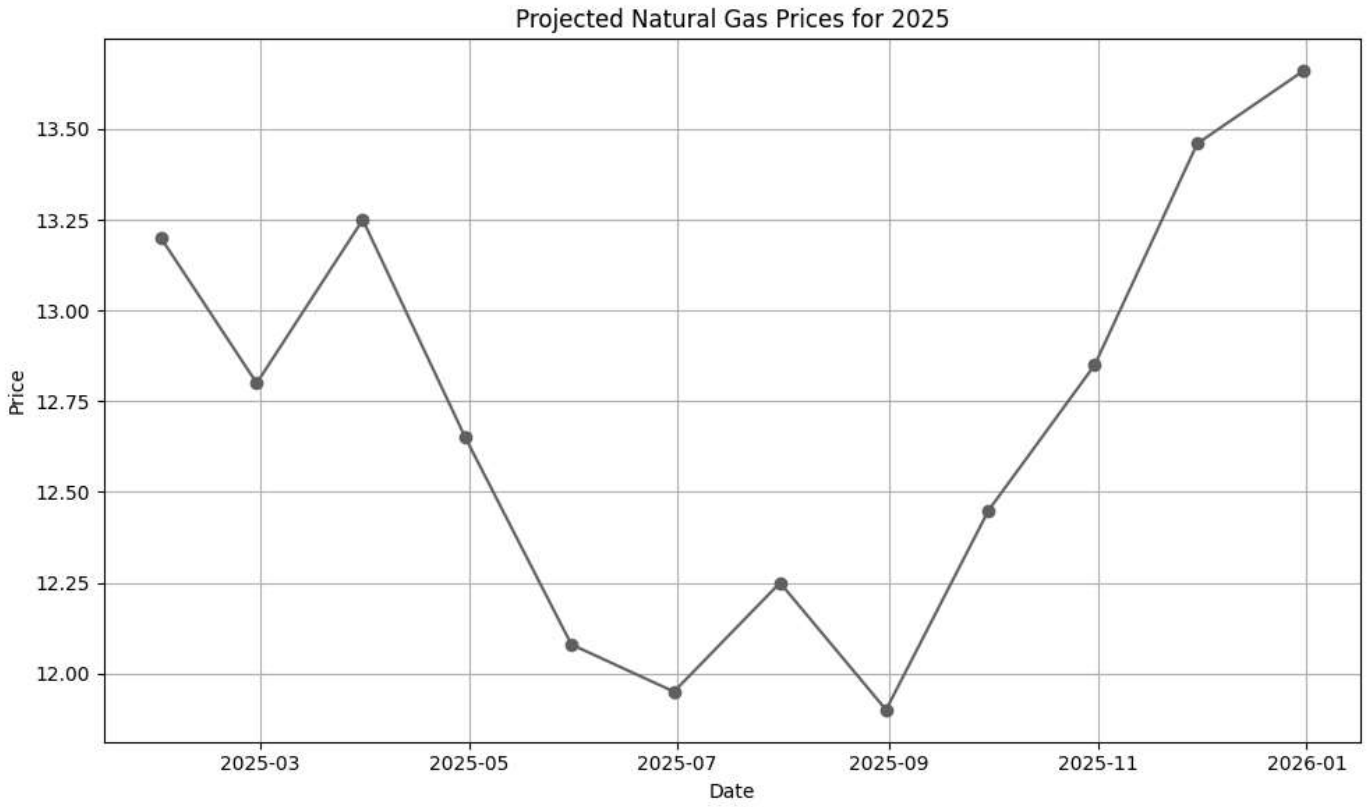
```
# New DAtaFrame of 2025 dates and prices
projected_gas_prices_2025_df = pd.DataFrame({'Dates': dates_2025, 'Prices': gas_price25})

# add moree coloumns
projected_gas_prices_2025_df['Dates'] = pd.to_datetime(projected_gas_prices_2025_df['Date
projected_gas_prices_2025_df['Year'] = projected_gas_prices_2025_df['Dates'].dt.year
projected_gas_prices_2025_df['Month'] = projected_gas_prices_2025_df['Dates'].dt.month

# show the DataFrame
projected_gas_prices_2025_df
```

Show hidden output

```
plt.figure(figsize=(10, 6))
plt.plot(projected_gas_prices_2025_df['Dates'], projected_gas_prices_2025_df['Prices'], r
plt.title('Projected Natural Gas Prices for 2025')
plt.xlabel('Date')
plt.ylabel('Price')
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
# Next year prediction ko add/concatinate kardo og datarae mein.

new_predicted_natural_gas_df_extrapolation_using_linear_reg = pd.concat([natural_gas_df_(

# add more coloumns
new_predicted_natural_gas_df_extrapolation_using_linear_reg['Dates'] = pd.to_datetime(new
new_predicted_natural_gas_df_extrapolation_using_linear_reg['Year'] = new_predicted_natur
new_predicted_natural_gas_df_extrapolation_using_linear_reg['Month'] = new_predicted_natu

# show the DataFrame
new_predicted_natural_gas_df_extrapolation_using_linear_reg
```

Show hidden output

Create a funtion to et the natural gas price for a gieven month and year

```
def get_gas_price(month, year):
  "Input month and year to get the price"
  print(new_predicted_natural_gas_df_extrapolation_using_linear_reg[ (new_predicted_natu
```

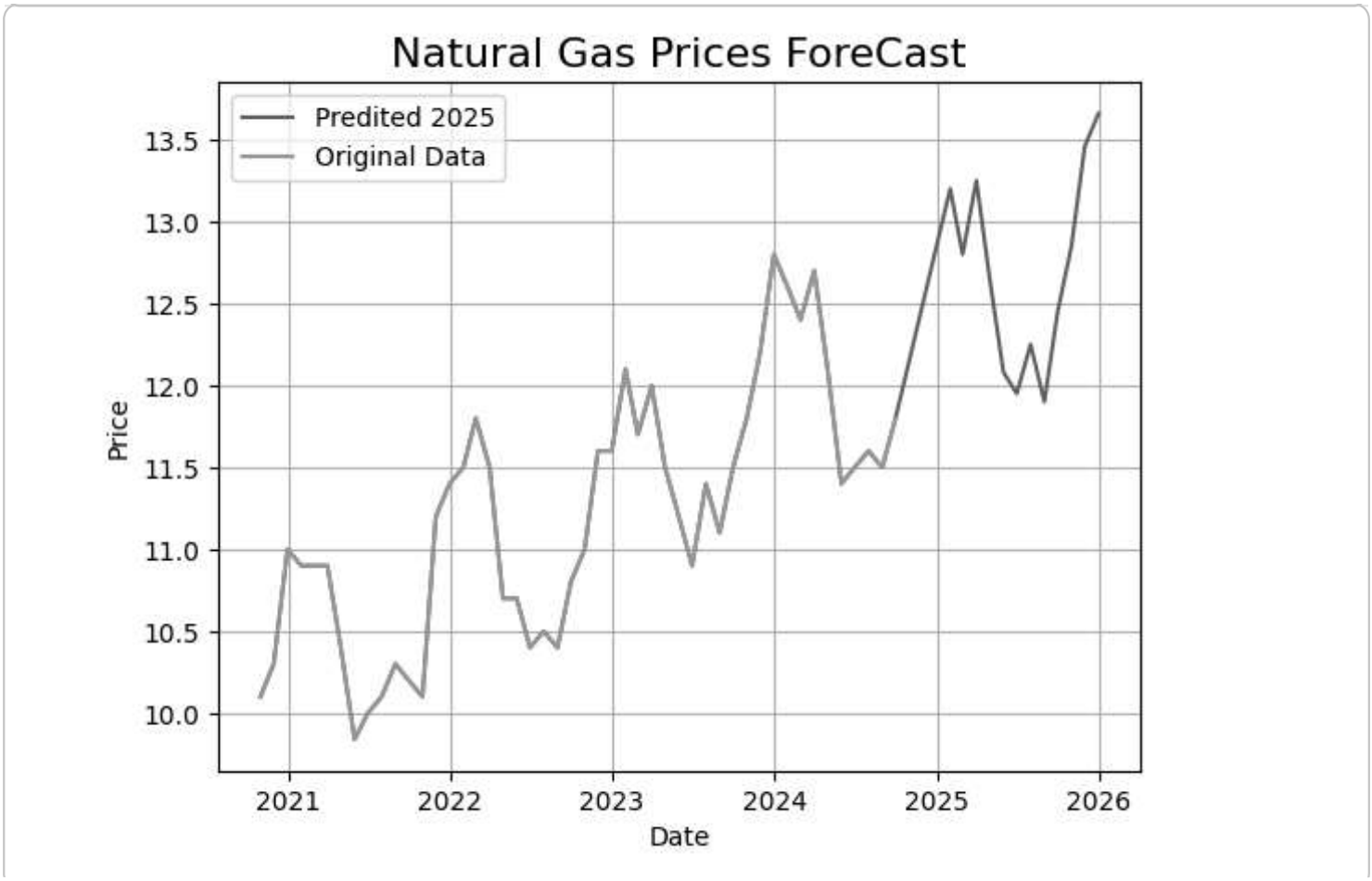
```
get_gas_price(3, 2025)
```

```
50    13.25
Name: Prices, dtype: float64
```

Final Analysis

```
# Prediced Values
plt.plot(new_predicted_natural_gas_df_extrapolation_using_linear_reg['Dates'], new_predi

# Actual Values
plt.plot(natural_gas_df_ORIGINAL['Dates'], natural_gas_df_ORIGINAL['Prices'], label = "O
plt.xlabel('Date')
plt.ylabel('Price')
plt.title("Natural Gas Prices ForeCast", fontsize=16)
plt.grid(True)
plt.legend()
plt.show()
```


Natural Gas Prices ForeCast
Predited 2025
Original Data
13.5
13.0
12.5
12.0
11.5
11.0
10.5
10.0
Price
2021
2022
2023
2024
2025
2026
Date